GitHub Version Control

Check this guide to learn more about version control using Git.

Git

Git is a distributed version control system (DVCS) that tracks changes in any set of computer files, usually used for coordinating work among programmers who are collaboratively developing source about during software development. It was originally authored in 2005 for the development of the Linux kernel.[1]

There are other version control tools and systems such as Mercurial , Subversion , Helix , Sapling , Bazaar , and Team Foundation Server , each with specific and general use cases. Git is the most popular and is integrated into almost every IDE out there. A comprehensive list of GUI clients is available here .

Commands

A selection of *porcelain* Git commands with corresponding tutorials to get started:

1. git init , git clone , git config ,git fetch , git pull , git add ,git commit , git push , git status , git diff , git rebase , git stash , git branch , git merge , git remote , git checkout, git reset , git revert , git clean , git rm , git log , git tag , git blame , git lfs

A cheat sheet of Git commands from Atlassian is available here .

Collaboration

1. Create a GitHub account, if you do not have one already, and sign up for the GitHub Student Developer Pack .
2. Install Git and GitHub Desktop , if needed.
3. Login with your GitHub credentials using GitHub Desktop/Visual Studio Code/IntelliJ IDEA.

Individual Repository

1. Join the GitHub Classroom invitation to create your individual *private* repository.
2. Clone [git clone] your individual repository locally using GitHub Desktop/Visual Studio Code/IntelliJ IDEA.
3. Stage [git add] your changes.
4. Commit [git commit] your staged changes locally using a descriptive and concise message.
5. Push [git push] your changes to sync them with the remote repository.
6. Use this process to practice and push your weekly exercises.

Team Repository

1. Join the GitHub Classroom invitation to create/join your team *private* repository.
2. Name your team repository using the following format team-section-teamname, for example, team-b56-plasma.
3. Clone [git clone] your team repository locally using GitHub Desktop/Visual Studio Code/IntelliJ IDEA.
4. Fetch [git fetch] to receive the latest changes.
5. Pull [git pull] to apply the changes to your working copy.
6. Stage [git add] your changes.
7. Commit [git commit) your staged changes locally using a descriptive and concise message.
8. Push [git push] your changes to sync them with the remote repository.
9. Use this process to collaborate on your project.

   You **must push** before deadlines as commit dates can be overridden using git commit --date="YYYY-MM-DD HH:MM:SS".

More info: Version Control · 251 OOP